Cybersecurity Intrusion Detection Using Machine Learning

Project explainer for team members · Course ECSCI24302 · generated 2026-10-28

1. What this project is

A supervised classifier that reads records of network traffic (network "flows") and predicts whether each one is **benign** or **malicious**. The dataset is **UNSW-NB15** (UNSW Canberra Cyber, 2015): real captured traffic plus synthetic modern attacks. The whole pipeline is deterministic (random_state=42) so every number is reproducible and identical across all files.

2. What has been produced (deliverables)

- A fully **executed Jupyter notebook** covering the complete pipeline end-to-end.
- A **project proposal** (DOCX) and a **final report** (DOCX + PDF) with chapters, figure captions and references.
- A **presentation deck** (PPTX) embedding the real figures generated by the pipeline.
- **Two browser frontends** for a live demo (Streamlit dashboard and a single-page HTML app).
- A **fitted model** (joblib) with a metadata file, so deploying needs no retraining.
- Unit tests (pytest) that exercise the pipeline.

3. The pipeline, step by step

Stage	What happens	Output
Load + validate	Rows, columns, id ranges, missing cells, label consistency are all re-verified at runtime.	dataset_facts.json
Quality audit	Counts duplicate flow rows and drops constants using training data only (results saved cleaned).	quality_audit.json
EDA	Class balance, feature histograms, box plots, correlation matrix (figures 1–7).	eda_summary.json
Split	Official training partition is the modeling source; the test partition stays untouched until the last step.	—
Feature engineering	16 derived features (totals, bytes/packet, send/receive ratios, loss rates, log1p versions, ftp/http flags).	inside pipeline
Preprocessing	Median imputer; StandardScaler only for the linear model; one-hot for proto/service/state.	inside pipeline
Baseline	DummyClassifier (most_frequent) sets the floor; never eligible for selection.	—
ML models	LogisticRegression, DecisionTree, RandomForest, HistGradientBoosting (identical pipelines).	—
Tuning	GridSearchCV / RandomizedSearchCV, cv=3, scoring=F1; full search spaces stored.	tuning_trials.json
Final test	One pass on the untouched test partition with the threshold fixed beforehand (0.5); metrics + figures (8–11).	—

4. Final results (single pass on the untouched test partition)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	AP
HistGradientBoosting (tuned)	0.8712	0.8154	0.9903	0.8944	0.9868	0.9903
RandomForest (tuned)	0.8663	0.8099	0.9894	0.8907	0.9841	0.9881
DecisionTree (tuned)	0.8601	0.8098	0.9894	0.8847	0.9693	0.9680
LogisticRegression (tuned)	0.8100	0.7550	0.9696	0.8490	0.9531	0.9650
Dummy (baseline floor)	0.5506	0.5506	1.0000	0.7102	0.5000	0.5506

5. Honest findings and limitations (say these in the viva)

- **The FPR is high (27.47%)** mainly because of a distribution shift between the official train and test partitions (68% vs 55% attack share). We do not patch it after the fact - we document it.
- **sttl dominates importance (0.375)** partly because it is a schema artifact of the synthetic capture; treat feature importance as indicative, never causal.
- **The release carries overlapping rows**: 4,247 duplicate flows between the two files (5.16%); F1 drops from 0.8944 to 0.8902 when only unique rows are scored.
- **Multi-class is harder than binary**: macro-F1 0.5129 across 9 families; Analysis (38.26%) and DoS (12.40%) recall are the weak spots.
- **Baseline never loses**: the dummy model shows the task is discriminative rather than trivial.
- **Operating point**: max F1 favours recall, matching the deployment brief ("catch attacks"); real environments re-calibrate per false-alarm cost.

6. Where everything lives in the code

- **src/data_loading.py** - loading, validation, DATASET_INFO, eda helpers.
- **src/preprocessing.py** - audit + FeatureEngineer + preprocessor factory.
- **src/evaluate.py** - metrics suite and all evaluation figures.
- **src/train.py** - the pipeline orchestrator and runbook.
- **src/predict.py** - the IntrusionPredictor used for fixed threshold inference.
- **app/app.py** (Streamlit) and **app/api.py** + **frontend/index.html** (HTML demo).
- **tests/test_pipeline.py** - pytest suite (5 tests, all pass).

7. Reproduce it yourself

pip install -r requirements.txt
then run **python -m src.download_data** (fetch + verify) and **python -m src.train** (about 6 minutes). All artefacts, figures and the results table regenerate deterministically (random_state=42). The model is benchmark-grade, not yet production-grade: re-calibrate the threshold against your own environment before using it anywhere real.